Đại học quốc gia TP.HCM
Trường đại học công nghệ thông tin

Khoa khoa học máy tính

Lớp: CS313.O21

BÀI TẬP CHƯƠNG 3

CHUẨN BỊ DỮ LIỆU

GV hướng dẫn: ThS. Nguyễn Thị Anh Thư

Nhóm sinh viên thực hiện

21521024 - Nguyễn Trung Kiên

21522792 - Phạm Quốc Việt

21520553 - Trần Hoài An

21522443 - Lường Đại Phát

22540017 - Phạm Minh Quốc

Mục lục

1 Yêu cầu bài tập **2**

2 Khám phá dữ liệu **2**

2.1 Thống kê mô tả 2

2.2 Trực quan hóa dữ liệu 3

2.3 Xử lý dữ liệu 4

2.3.1 Xử lý dữ liệu thiếu 4

2.3.2 Xử lý giá trị nhiễu 6

2.4 Phân tích thống kê 9

2.4.1 Kiểm định t-test 9

2.4.2 Kiểm định Anova 10

2.4.3 Kiểm định Mann-Whitney U 11

2.4.4 Kiểm định tương đồng Jaccard 12

2.4.5 Mô hình hồi quy 14

2.4.6 Hệ số tương quan Spearman 16

2.5 Khai phá tri thức 17

2.5.1 Luật kết hợp 17

2.5.2 Đo lường luật kết hợp 18

2.5.3 Thuật toán Apriori 19

3 Làm sạch dữ liệu **23**

3.1 Xử lý dữ liệu trùng lặp 23

4 Tài liệu tham khảo **26**

1 Yêu cầu bài tập

Tìm hiểu công cụ và thư viện hỗ trợ Python, bao gồm:

- Công dụng, cú pháp sử dụng, ví dụ demo cách áp dụng và vận dụng vào tập dữ liệu khai thác của nhóm.
- Các phần cần thực hiện bao gồm khám phá dữ liệu (thống kê mô tả, trực quan hóa dữ liệu, xử lý dữ liệu, phân tích thống kê, khai phá tri thức) và làm sạch dữ liệu (xử lý dữ liệu trùng lặp).

2 Khám phá dữ liệu

2.1 Thống kê mô tả

Với các đặc trưng số, thống kê mô tả là việc khảo sát các giá trị của một thuộc tính như: trung bình, trung vị, độ lệch chuẩn, tứ phân vị. Ví dụ, trong file user-problem.json có một thuộc tính tên là *attempts* - số lần thử lại cho câu hỏi của người học.

Thống kê trên cột *attempts* với 10000 dòng đầu tiên của user-problem.json bằng cách sử dụng các thuộc tính được cho sẵn bởi thư viện Pandas.

```
user_problem_df = pd.read_json("relations/user-problem.json", lines = True, nrows = 10000)
```

Thống kê lần lượt giá trị nhỏ nhất, giá trị lớn nhất, trung bình, trung vị, độ lệch chuẩn và tứ phân vị của cột attempts bằng cách sử dụng Pandas

```
mean_attempts = user_problem_df['attempts'].mean() # giá trị trung bình
median_attempts = user_problem_df['attempts'].median() # giá trị trung vị
std_attempts = user_problem_df['attempts'].std() # độ lệch chuẩn
quantiles_attempts = user_problem_df['attempts'].quantile([0.25, 0.5, 0.75]) # tứ phân vị
```

```
# in kết quả
print("Giá trị min, max của cột 'attempts': ", user_problem_df['attempts'].min(), user_problem_df['attempts'].max())
print("Giá trị trung bình của cột 'attempts': ", mean_attempts)
print("Trung vị của cột 'attempts':", median_attempts)
print("Độ lệch chuẩn của cột 'attempts'", std_attempts)
print("Tứ phân vị của cột 'attempts'")
print(quantiles_attempts)
```

```
Giá trị min, max của cột 'attempts':  1 121
Giá trị trung bình của cột 'attempts':  1.857
Trung vị của cột 'attempts': 1.0
Độ lệch chuẩn của cột 'attempts' 4.862809444299577
Tứ phân vị của cột 'attempts'
0.25    1.0
0.50    1.0
0.75    1.0
Name: attempts, dtype: float64
```

Hình 1: Code và output các giá trị nhỏ nhất, lớn nhất, trung bình, trung vị, độ lệch chuẩn và tứ phân vị của cột *attempts*

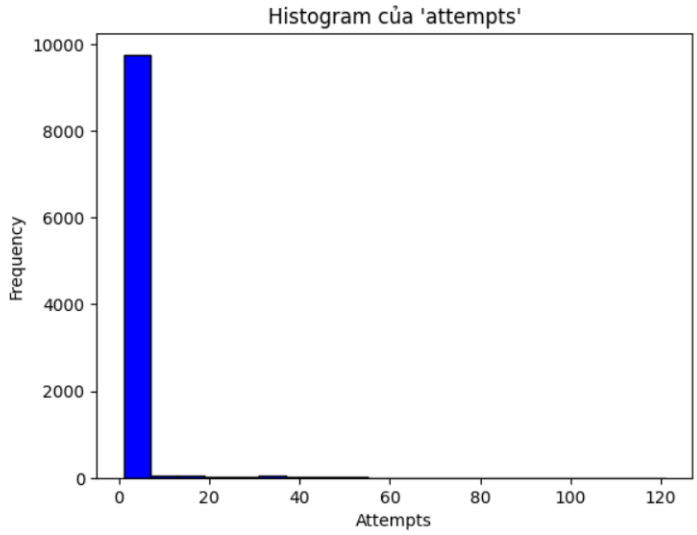
Ta có thể thực hiện việc thống kê các giá trị kể trên bằng các câu lệnh tương tự trên các **đặc trưng số**. Xét riêng trên 10000 giá trị đầu của cột *attempts*, các số liệu ở hình 1 chỉ ra các đặc điểm bao gồm:

- Số lần thử tối thiểu cho một vấn đề là 1.
- Số lần thử tối đa cho một vấn đề là 121.
- Trung bình một người học cần 1.857 lần thử cho một vấn đề.
- Độ lệch chuẩn của cột là 4.863.
- Q1, Q2, Q3 (các điểm tứ phân vị) đều bằng 1. Dễ dàng nhận thấy các giá trị '1' trong 10000 dòng đầu chiếm đa số, hay đa số các vấn đề đều chỉ cần thử 1 lần duy nhất.

2.2 Trực quan hóa dữ liệu

Trực quan hóa dữ liệu là việc thực hiện phân tích phân phối dữ liệu bằng cách sử dụng biểu đồ các biểu đồ như histogram, boxplot, ...

```
plt.hist(user_problem_df['attempts'], bins = 20, color = 'blue', edgecolor = 'black')
plt.title("Histogram của 'attempts'")
plt.xlabel('Attempts')
plt.ylabel('Frequency')
plt.show()
```



Hình 2: Code và biểu diễn đồ thị histogram của thuộc tính *attempts*

Trong phần 2.1, khi quan sát thấy các điểm tứ phân vị Q1, Q2, Q3 đều là 1, nhóm đưa ra khẳng định các giá trị '1' trong 10000 dòng đầu chiếm đa số. Nhóm thực hiện trực quan hóa dữ liệu trên cột *'attempts'* để chứng minh khẳng định trên.

Trực quan dữ liệu lên histogram sử dụng *matplotlib.pyplot* (hình 2) đã khẳng định được được số dòng giá trị có *attempts = 1* lên đến gần 10000 (thể hiện bởi cột màu xanh).

2.3 Xử lý dữ liệu

2.3.1 Xử lý dữ liệu thiếu

Là bước xử lý cho các giá trị N/A trong tập dữ liệu. Có 3 cách để xử lý bao gồm:

- Xóa bỏ: Xóa các điểm dữ liệu có quá nhiều giá trị thiếu.
- Thay thế: Thay thế giá trị thiếu bằng trung bình, trung vị, hoặc giá trị phổ biến nhất của cột dữ liệu.
- Dự đoán: Sử dụng các kỹ thuật học máy để dự đoán giá trị thiếu dựa trên các dữ liệu có sẵn.

Quy trình xử lý dữ liệu mà nhóm đề ra như sau:

- Với các trường dữ liệu số, nhóm lựa chọn phương pháp thay thế các giá trị N/A bằng giá trị trung bình của cột đó (với dữ liệu float) hay giá trị trung vị (với dữ liệu int).
- Với các trường dữ liệu không phải số, nhóm lựa chọn giải pháp xóa bỏ, trừ khi phải giữ nguyên vì có thể nếu xóa bỏ thì các dữ liệu quan trọng có thể bị mất (1 giá trị N/A phải khiến cả dòng xóa bỏ thì dễ gây mất mát dữ liệu và tạo sự khó dễ trong việc truy vấn dữ liệu nếu cần thiết).

Trước hết, ta cần biết liệu một cột trong dataframe có dòng nào chứa giá trị N/A hay không. Sử dụng lệnh *df.isna().sum()* với *df* là dataframe cần xét để xem số lượng giá trị N/A của từng cột trong *df*. Nhóm thử thực hiện trên tập dữ liệu video.json.

```
video_df = pd.read_json("entities/video.json", lines = True)
```

```
video_df
```

	ccid	name	start	end	text
0	0001603F826A3DDA9C33DC5901307461	Video	[1.031, 7.095, 8.935, 10.279, 14.759, 15.977, ...	[4.255, 8.119, 10.033, 14.327, 15.751, 18.807,...	[第二个就是短助记符在生成上面。有一个规定, 短助记符是这样说, 就说它最多是四个, 如果长...
1	0003DB14A14A53279C33DC5901307461	Video	[11.799, 12.581, 17.38, 19.765, 22.257, 25.215...	[12.581, 17.38, 19.765, 22.257, 25.215, 29.601...	[大家好, 今天我们讨论一维系统辐射传递一般近似解, 我们推导出辐射传递方程后, 讨论了一维...
2	0004A5C6F07E369D9C33DC5901307461	家禽的消化系统	[19.2, 20.84, 23.4, 26.48, 27.796, 36.189, 40....	[20.83, 23.39, 26.47, 27.786, 36.179, 40.19, 4...	[各位同学 大家好, 我是来自中国农业大学的曹静, 今天我们来学习家禽的解剖特征, 家禽的消...
3	00059EBD1371A6959C33DC5901307461	1.2 计算机硬件系统	[19.372, 190.522, 194.572, 438.838, 540.972, 5...	[23.14, 194.458, 198.084, 442.037, 542.564, 54...	[大家好 本视频介绍计算机硬件系统, 总线就其类别来说 在计算机系统当中包括, 内部总线 ...
4	0005D1DC01B4EF8F9C33DC5901307461	什么是供应链网络	[24.33, 25.081, 26.775, 27.271, 28.759, 30.511...	[25.08, 26.775, 27.27, 28.759, 30.51, 31.85, 3...	[大家好, 今天我们学习的知识点, 是什么, 是供应链网络, 跟大家举两个例子, 京东的亚洲...
...	...	...	...	...	...
59576	FFF93B8BC78D14B29C33DC5901307461	2.4临床试验人群选择的微创新	[7.47, 9.4, 12.32, 15.17, 17.73, 20.65, 22.29,...	[9.22, 12.16, 14.76, 17.69, 20.65, 22.25, 25.0...	[同学们 大家好, 这节我们将集中分析案例, 讨论人群选择的微创新, 在上一章选题思路的部分...
59577	FFF99B018E97FA6E9C33DC5901307461	2.3-2 Presentation of Sound and Graphics Infor...	[4.203, 7.702, 10.496, 14.603, 17.642, 19.53, ...	[7.468, 10.013, 14.41, 16.317, 19.378, 22.54, ...	[Next. I will briefly talk about the image, ab...
59578	FFFB9B63E30B25919C33DC5901307461	Video	[18.44, 21.36, 25.64, 29.32, 33.24, 35.72, 38....	[21.36, 25.64, 29.32, 33.24, 35.72, 38.64, 41....	[现在开始岩石和集料第二讲, 岩石的其他性质和岩石的技术等级, 首先我们讨论岩石的耐久性, ...
59579	FFFE61FFCECC05729C33DC5901307461	12.3 物料清单	[21.96, 26.63, 30.425, 31.77, 34.53, 36.84, 38...	[26.06, 30.425, 30.92, 34.26, 36.58, 38.47, 40...	[物料清单是对产品结构的一个描述, 英文的说法叫Bill of Materials, 简称B...
59580	FFFF3CC4C6F66F849C33DC5901307461	2.44	[28.6, 31.8, 33.225, 36.025, 38.2, 42.975, 46....	[30.725, 32.975, 35.4, 37.675, 41.45, 44.475, ...	[第二个能力是洞察力, 善于创新的人, 身上都有惊人的洞察力, 他们能洞察细节, 能看到问题...

59581 rows × 5 columns

```
print(video_df.isna().sum())
```

```
ccid     0
name     0
start    0
end      0
text     0
dtype: int64
```

Hình 3: Dữ liệu của video.json và số liệu các giá trị N/A trên từng cột của video.json. Không có cột nào chứa giá trị N/A trong tập dữ liệu

Xét trên 1 file khác là user.json. Từ hình 4, ta thấy có cột *year_of_birth* thuộc kiểu dữ liệu số nguyên nhưng lại có 8229 giá trị N/A.

```
user_df = pd.read_json("entities/user.json", lines = True, nrows = 10000)
```

```
print(user_df.isna().sum())
```

```
id                 0
name               0
gender             0
school             0
year_of_birth   8229
course_order       0
enroll_time        0
dtype: int64
```

Hình 4: Số liệu các giá trị N/A trên từng cột của user.json.

Nhóm thực hiện việc xử lý giá trị thiếu bằng cách điền vào các vị trí còn thiếu giá trị trung vị của cột, ta dùng lệnh *df['x'].fillna(y, inplace = True)* với x là tên cột thực hiện, y là giá trị thực hiện. Do đó, câu lệnh cần thực hiện sẽ là: data['year_of_birth'].fillna(data['year_of_birth'].median(), inplace=True).

Kết quả thu được khi áp dụng câu lệnh ở hình 5.

```
user_df['year_of_birth'].fillna(user_df['year_of_birth'].median(), inplace = True)
```

```
print(user_df['year_of_birth'])
```

```
0       2015.0
1       6558.0
2       1997.0
3       1973.0
4       1997.0
         ...
9995    1997.0
9996    1998.0
9997    1997.0
9998    1997.0
9999    1997.0
Name: year_of_birth, Length: 10000, dtype: float64
```

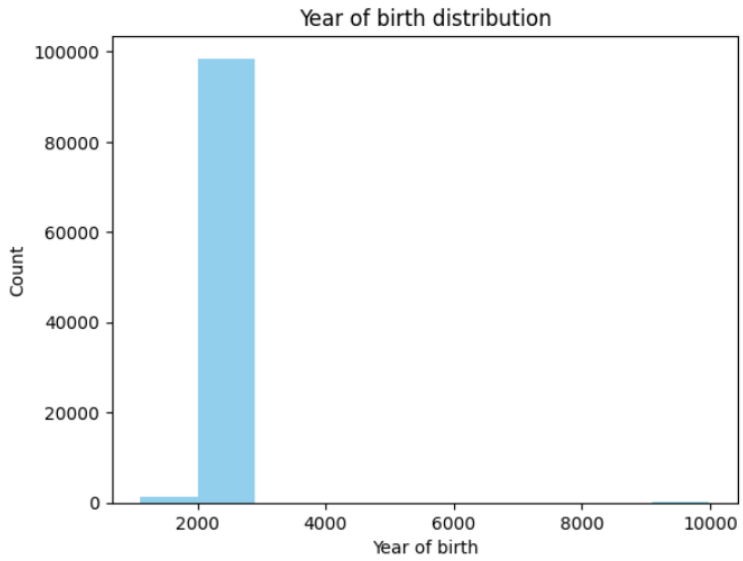
Hình 5: Dữ liệu của user.json trên 10000 dòng đầu sau khi đã điền giá trị trung vị 1997.0 vào các vị trí còn thiếu trên cột *year_of_birth*

2.3.2 Xử lý giá trị nhiễu

Trong các trường dữ liệu dạng số, ta thường bắt gặp các giá trị ngoại lai gây nhiễu, làm ảnh hưởng đến phân phối của dữ liệu. Ta cần xử lí những giá trị này bằng một số cách như:

- Xóa bỏ: Loại bỏ các quan sát có giá trị nhiễu khỏi tập dữ liệu.
- Thay thế: Thay thế giá trị nhiễu bằng trung bình, trung vị, hoặc giá trị phổ biến nhất của cột dữ liệu.
- Dự đoán: Sử dụng các kỹ thuật học máy để dự đoán giá trị nhiễu từ các giá trị khác.
- Chuẩn hóa: Chuẩn hóa dữ liệu để giảm thiểu ảnh hưởng của nhiễu bằng cách chia tỷ lệ dữ liệu thành khoảng giá trị nhất định hoặc chuyển đổi các giá trị thành phân phối chuẩn.

Để biết trường dữ liệu của một dataframe có xuất hiện nhiễu hay không, ta cần quan sát phân phối của trường dữ liệu đó. Ví dụ, ta sẽ xét 100000 dòng đầu tiên của trường *year_of_birth* của tập dữ liệu user.json.



Hình 6: Phân phối của cột *year_of_birth*

Có thể thấy giá trị của trường year_of_birth chủ yếu rơi vào khoảng 2000, một số giá trị nhiễu như 10000 gây ảnh hưởng đến phân phối của trường dữ liệu.

```
# Các lệnh thống kê mô tả ở phần 2.1.1 có thể dùng một phương thức .describe() để hiển thị
user_df.describe()
```

	gender	year_of_birth
count	10000.000000	10000.000000
mean	0.917500	2078.890600
std	0.797971	717.688979
min	0.000000	1111.000000
25%	0.000000	1997.000000
50%	1.000000	1997.000000
75%	2.000000	1997.000000
max	2.000000	9989.000000

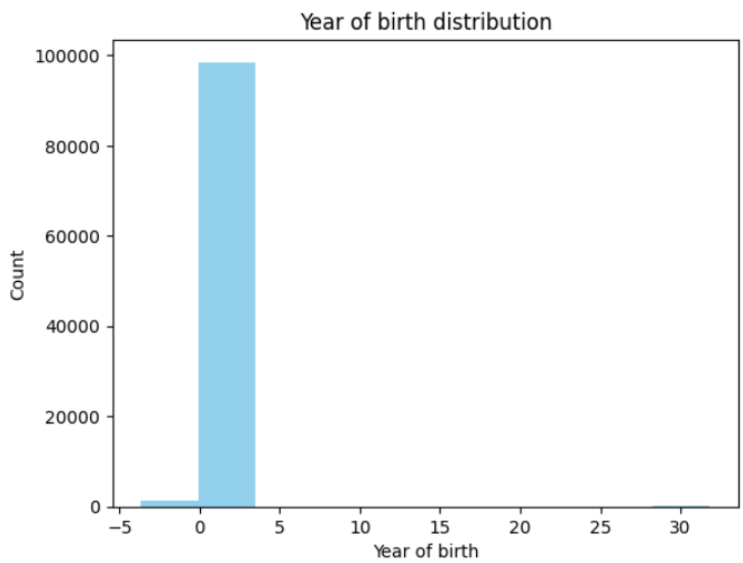
Hình 7: Các giá trị thống kê của trường *year_of_birth*, *.describe()* là phương thức all-in-one để hiển thị các giá trị thống kê ở phần 2.1

Để giảm thiểu sự ảnh hưởng của các giá trị nhiễu lên phân phối, ta áp dụng kĩ thuật chuẩn hóa lên trường dữ liệu year_of_birth, cụ thể là StandardScaler.

```
[77] scaler = StandardScaler()
     new_user = user.loc[:,['year_of_birth']]
     new_user = scaler.fit_transform(new_user)
```

```
[88] scaled_user = pd.DataFrame(new_user, columns=['year_of_birth'])
```

Hình 8: Tiến hành chuẩn hóa trường *year_of_birth*



Hình 9: Phân phối của trường *year_of_birth* sau khi chuẩn hóa

Ta có thể thấy độ lệch chuẩn đã giảm đáng kể, miền giá trị cũng được phân phối đều hơn sau khi chúng ta áp dụng phương pháp chuẩn hóa.

```
[91] scaled_user.describe()
```

	year_of_birth
count	1.000000e+05
mean	-5.215384e-17
std	1.000005e+00
min	-3.654748e+00
25%	-3.744753e-02
50%	-3.744753e-02
75%	-3.744753e-02
max	3.183077e+01

Hình 10: Giá trị thống kê của trường *year_ of_ birth* sau khi chuẩn hóa

2.4 Phân tích thống kê

Thực hiện các bài kiểm định thống kê để xác định mối quan hệ thống kê giữa các biến để có thể giúp hiểu rõ về cấu trúc, tính chất và đặc điểm của dữ liệu, bao gồm phân phối, độ biến thiên, giá trị ngoại lệ và sự tương quan giữa các biến.

2.4.1 Kiểm định t-test

Kiểm định t (t-test) là một phương pháp thống kê được sử dụng để so sánh trung bình của hai nhóm dữ liệu. Đây là một trong những công cụ phổ biến nhất trong thống kê để kiểm tra sự khác biệt giữa các trung bình mẫu.

- Mô tả: kiểm tra xem có sự khác biệt về năm sinh trung bình giữa hai nhóm sinh viên nam và nữ không.
- Thư viện: *scipy*
- Phương thức: *from scipy.stats import ttest_ ind*
- Input: Thuộc tính kiểm định *gender* và thuộc tính mục tiêu *year_ of_ birth*
- Output: Kết quả được in ra bao gồm giá trị thống kê *t* và *p-value*. Nếu *p-value* nhỏ hơn một ngưỡng ý nghĩa (thường là 0.05), chúng ta có thể bác bỏ giả thuyết không có sự khác biệt giữa hai nhóm.

```python
from scipy import stats
from scipy.stats import ttest_ind
```

```python
user_df = pd.read_json('entities/user.json', lines = True, nrows = 1e5)
```

```python
df = pd.DataFrame(user)

# Loại bỏ các giá trị null trong cột year_of_birth và lấy ra dữ liệu của nam và nữ
male_birth_year = df.loc[df['gender'] == 1, 'year_of_birth'].dropna()
female_birth_year = df.loc[df['gender'] == 0, 'year_of_birth'].dropna()

# Thực hiện kiểm định t-test độc lập
t_statistic, p_value = ttest_ind(male_birth_year, female_birth_year)

# In kết quả
print("Giá trị thống kê t:", t_statistic)
print("p-value:", p_value)
if p_value < 0.05:
    print("Có đủ bằng chứng để bác bỏ giả thuyết không có sự khác biệt về năm sinh trung bình giữa nam và nữ.")
else:
    print("Không có đủ bằng chứng để bác bỏ giả thuyết không có sự khác biệt về năm sinh trung bình giữa nam và nữ.")
```

```
Statistic: -16.294849677126013
p-value: 4.539252670193533e-59
```

Hình 11: Kiểm định t-test

p-value = 4.5392e-59 cho ta biết rằng ta có thể phủ định việc không có sự khác biệt về năm sinh trung bình giữa nam và nữ, hay có thể khẳng định là có sự khác biệt đáng kể về năm sinh trung bình giữa hai nhóm nam và nữ.

2.4.2 Kiểm định Anova

ANOVA Test khá tương tự T-Test, tuy nhiên T-Test chỉ kiểm định được cho thuộc có hai nhóm quan sát, còn anova kiểm định được cho thuộc tính có từ 2 nhóm quan sát trở lên:

- Mô tả: Kiểm định sự khác biệt về năm sinh trung bình giữa các trường.
- Thư viện: *scipy*
- Phương thức: *stats.f_oneway*
- Input: Thuộc tính kiểm định *school* và thuộc tính mục tiêu *year_of_birth*
- Output: Cặp giá trị *F-statistic* và *p-value* cho biết sự khác biệt giữa các nhóm quan sát và độ tin cậy của phương pháp đánh giá (*p-value* < 0.05 cho thấy kiểm định có ý nghĩa thống kê, giá trị *F-statistic* càng cao càng thể hiện sự khác biệt giữa các nhóm quan sát)

```python
x = user_df.loc[user_df['year_of_birth'].notna()] #loại bỏ các dòng dữ liệu chứa
#giá trị NaN trên thuộc tính "year_of_birth" nhằm thực hiện nhanh demo cho kiểm định ANOVA
def anova_table(df, categorical_columns, numerical_column):
    result_table = pd.DataFrame(columns=['Category', 'F-Statistic', 'p-value'])

    for cat_col in categorical_columns:
        # Group data by categories
        grouped_data = df.groupby(cat_col)[numerical_column].apply(list)

        # Perform Anova test
        f_statistic, p_value = stats.f_oneway(*grouped_data)

        # Append results to the result table
        result_table = pd.concat([result_table, pd.DataFrame.from_dict
         ({'Category': [cat_col],'F-Statistic': [f_statistic], 'p-value': [p_value]})], ignore_index=True)
    return result_table

selected_columns = ['gender', 'school']
result_anova = anova_table(x, selected_columns, 'year_of_birth')
result_anova
```

	Category	F-Statistic	p-value
0	gender	26.471870	3.885312e-12
1	school	1.011423	4.153299e-01

Hình 12: Kiểm định Anova

2 giá trị p-value trên rất nhỏ (<0.05) nên chúng ta có thể kết luận rằng có sự khác biệt đáng kể về năm sinh trung bình giữa các trường khác nhau. Điều này ngụ ý rằng ít nhất một cặp trường có sự khác biệt đáng kể trong năm sinh trung bình của sinh viên.

2.4.3 Kiểm định Mann-Whitney U

Kiểm định Mann-Whitney U (còn được gọi là kiểm định U của Mann-Whitney hoặc kiểm định U của Wilcoxon) là một phương pháp thống kê phi tham số được sử dụng để kiểm tra sự khác biệt giữa hai nhóm độc lập trong một biến rời rạc.

- Mô tả: kiểm tra xem có sự khác biệt đáng kể về năm sinh trung bình giữa nam và nữ hay không
- Thư viện: *scipy*
- Phương thức: *from scipy.stats import mannwhitneyu*
- Input: Thuộc tính kiểm định *gender* và thuộc tính mục tiêu *year_of_birth*

- Output: Kết quả được in ra bao gồm giá trị thống kê U và *p-value*. Nếu *p-value* nhỏ hơn một ngưỡng ý nghĩa (thường là 0.05), chúng ta có thể bác bỏ giả thuyết không có sự khác biệt giữa hai nhóm.

```
from scipy.stats import mannwhitneyu
df = pd.DataFrame(user)

# Loại bỏ các dòng có giá trị year_of_birth là null
df = df.dropna(subset=['year_of_birth'])

# Tách dữ liệu thành hai nhóm: nam và nữ
male_birth_year = df.loc[df['gender'] == 1, 'year_of_birth']
female_birth_year = df.loc[df['gender'] == 0, 'year_of_birth']

# Thực hiện kiểm định Mann-Whitney U
statistic, p_value = mannwhitneyu(male_birth_year, female_birth_year)


# In kết quả
print("Giá trị thống kê U:", statistic)
print("p-value:", p_value)
if p_value < 0.05:
    print("Có đủ bằng chứng để bác bỏ giả thuyết không có sự khác biệt giữa năm sinh trung bình của nam và nữ.")
else:
    print("Không có đủ bằng chứng để bác bỏ giả thuyết không có sự khác biệt giữa năm sinh trung bình của nam và nữ.")
```

```
Statistic: 452519720.0
p-value: 9.205717750914102e-64
```

Hình 13: Kiểm định Mann-Whitney U

p-value = 9.2-571e-64 cho ta biết rằng ta có thể phủ định việc không có sự khác biệt về năm sinh trung bình giữa nam và nữ, tức có sự khác biệt đáng kể về năm sinh trung bình giữa hai nhóm nam và nữ.

2.4.4 Kiểm định tương đồng Jaccard

Kiểm định tương đồng Jaccard là một phương pháp được sử dụng để đánh giá mức độ tương đồng giữa hai tập hợp dữ liệu. Ví dụ với 2 tập hợp dữ liệu A và B, ta có công thức tính Jaccard similarity như sau:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Trong đó:

- $|A \cap B|$ là số lượng phần tử chung giữa hai tập hợp A và B.

- $|A \cup B|$ là tổng số lượng phần tử của hai tập hợp A và B (bao gồm cả các phần tử trùng lặp).

J(A, B) nằm trong khoảng [0, 1], càng gần 0 cho thấy A và B không có nhiều phần tử chung (có sự khác biệt lớn và ngược lại, càng gần 1 cho thấy A và B rất tương đồng với nhau). Trong bài này, nhóm sử dụng độ đo Jaccard similarity để đánh giá độ tương đồng trung bình trong thứ tự khóa học mà các sinh viên cùng 1 trường đã học.

- Input: Dữ liệu các thuộc tính *school* và *course_order* được lưu dưới dạng các đối tượng Series.
- Output: Độ tương đồng Jaccard similarity về thứ tự khóa học của các sinh viên trong cùng 1 trường.

```
def compute_jaccard_similarity (course_list1, course_list2):
    set1 = set(course_list1)
    set2 = set(course_list2)
    intersection = len (set1.intersection(set2))
    union = len(set1.union(set2))
    jaccard_sim = intersection/union if union != 0 else 0
    return jaccard_sim
```

```
schools = x['school'].unique()
cos_means = []
jac_means = []

for school in schools:
    cos_list = []
    jac_list = []
    df_school = x[x['school'] == school]
    random_samples = df_school.sample(n = 10, replace = True)

    for i in range(len(random_samples) - 1):
        for j in range (i+1, len(random_samples)):
            user1 = random_samples.iloc[i]['course_order']
            user2 = random_samples.iloc[j]['course_order']

            jac_list.append(compute_jaccard_similarity(user1, user2))

    jac_means.append(np.mean(jac_list))

np.mean(jac_means)
```

```
0.74778208778103
```

Hình 14: Kiểm định tương đồng Jaccard similarity

Giá trị Jaccard similarity là 0.74778, cho thấy thứ tự khóa học mà các sinh viên cùng 1 trường đã học là khá tương đồng với nhau.

2.4.5 Mô hình hồi quy

"Mô hình" có thể hiểu là một hàm f sao cho với input đã có, hàm f cho ta dự đoán về output tương ứng với input, hay f(input) = output dự đoán bởi f. Nếu f là một "mô hình hồi quy" thì f có thể được biểu diễn dưới dạng một phương trình hồi quy tuyến tính. Mục tiêu của việc xác định một mô hình hồi quy là vì từ nó, ta có thể xác định được các phân tích thống kê cần thiết.

Nhóm thực hiện phân tích thống kê bằng mô hình hồi quy trên user.json, xét cột dữ liệu *year_of_birth* là input, *gender* là output bằng cách sử dụng thư viện: *statsmodels.api* - một thư viện cung cấp một loạt các mô hình thống kê và kiểm định, bao gồm hồi quy tuyến tính, mô hình tuyến tính tổng quát, ... Một vài các thống kê có thể xác định từ mô hình hồi quy tuyến tính như:

- Hệ số tương quan R^2: có giá trị nằm trong đoạn [0, 1], đánh giá mức độ dự đoán của mô hình tốt đến đâu cho dữ liệu out-of-sample. Với ví dụ thực hiện, R^2 sẽ cho biết dựa vào dữ liệu của *year_of_birth* có thể đoán đúng được *gender* đến mức nào.

- AIC (Akaike Information Criterion): AIC đánh giá sự "khớp" của một mô hình với dữ liệu và có penalty cho lượng tham số trong mô hình. AIC càng nhỏ thì mô hình được coi là càng tốt.

- BIC (Bayesian Information Criterion): BIC cũng đo lường mức độ khớp của một mô hình với dữ liệu và có penalty cho số lượng tham số trong mô hình. Tuy nhiên, penalty của BIC cao hơn so với AIC.

```
!pip install statsmodels
import statsmodels.api as sm

# Khởi tạo và khớp mô hình
model = sm.OLS(user_df['year_of_birth'], user_df['gender'])
results = model.fit()
```

Hình 15: Fit dữ liệu vào mô hình hồi quy tuyến tính (OLS)

Trong thực tế, ngày sinh thì không tác động được vào giới tính của một con người, nên ta sẽ trông chờ giá trị R^2 xét trên 2 thuộc tính *year_of_birth* và *gender* thấp, còn 2 giá trị AIC và BIC cao.

```
# In kết quả
print(results.summary())
```

```
                                 OLS Regression Results
=======================================================================================
Dep. Variable:          year_of_birth   R-squared (uncentered):                  0.524
Model:                            OLS   Adj. R-squared (uncentered):             0.524
Method:                 Least Squares   F-statistic:                         1.100e+04
Date:                Wed, 20 Mar 2024   Prob (F-statistic):                       0.00
Time:                        01:48:03   Log-Likelihood:                         -87438.
No. Observations:               10000   AIC:                                 1.749e+05
Df Residuals:                    9999   BIC:                                 1.749e+05
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
gender      1309.1260     12.481    104.889      0.000    1284.661    1333.591
==============================================================================
Omnibus:                     3236.183   Durbin-Watson:                   1.267
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            22847.186
Skew:                           1.368   Prob(JB):                         0.00
Kurtosis:                       9.881   Cond. No.                         1.00
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a constant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Hình 16: Phân tích thống kê từ mô hình hồi quy tuyến tính

Kết quả cho thấy giá trị R^2 là 0.524, đồng nghĩa với việc *year_of_birth* không phải là một thuộc tính ảnh hưởng lớn với *gender*, 2 giá trị AIC và BIC đều lớn (>1e5) cho thấy với mô hình hồi quy tuyến tính tốt nhất khi fit vào dữ liệu thì đây cũng không phải là một mô hình quá tốt.

Giá trị t và p trong output chính là giá trị thống kê t và *p_value* trong kiểm định t-test. Giá trị t từ output có giá trị 104.889 và P > |t| là 0.000, điều này cho thấy có sự khác biệt lớn về những giới tính dự đoán với những năm sinh. Ví dụ như những người sinh năm 2000 có những giới tính khác nhau, không phải người này sinh năm 2000 và có giới tính nam thì đa số những người khác cũng là nam giới. Đó là một số những yếu tố nhóm liệt kê từ kết quả của mô hình hồi quy.

2.4.6 Hệ số tương quan Spearman

Được sử dụng khi hai biến x và y không tuân theo phân phối chuẩn, trái ngược với hệ số tương quan Pearson. Hệ số này được ước tính bằng cách biến đổi biến 2 thuộc tính thành thuộc tính có thứ bậc (rank), sau đó xem xét độ tương quan của 2 rank đó.

- Mô tả: Đo lường mối quan hệ thống kê hoặc liên kết giữa các biến phụ thuộc *year_of_birth* với các biến mục tiêu *course_count*.
- Thư viện: *scipy*
- Phương thức: *stats.spearmanr*
- Input: Dữ liệu của thuộc tính phụ thuộc và thuộc tính mục tiêu được lưu dưới dạng đối tượng kiểu Series
- Output: Cặp giá trị *statistic* và *p-value* cho biết mối tương quan giữa 2 thuộc tính và độ tin cậy của phương pháp đánh giá (p-value < 0.05 cho thấy đánh giá có ý nghĩa thống kê)

```
#thêm thuộc tính "course_count" vào bộ dữ liệu đã loại bỏ các dòng dữ liệu chứa giá trị NaN trên thuộc tính "year_of_birth"
x['course_count'] = x['course_order'].str.len()
```

```
import scipy
def corr2(df):
  selected_columns = df[['year_of_birth']]
  result_table = pd.DataFrame(columns=['Column', 'pearson-r', 'p-value', 'Tương Quan'])
  for col in selected_columns.columns:
    r, p = scipy.stats.spearmanr(df[col],df['course_count'])
    if p<0.05:
      if(0.8<r<=1) or (-1<=r<-0.8):
        result_table = pd.concat([result_table, pd.DataFrame.from_dict
         ({'Column': [col], 'pearson-r': [r], 'p-value': [p], 'Tương Quan': ['Mạnh']})], ignore_index=True)
      elif(0.5<r<=0.8) or (-0.8<=r<-0.5):
        result_table = pd.concat([result_table, pd.DataFrame.from_dict
         ({'Column': [col], 'pearson-r': [r], 'p-value': [p], 'Tương Quan': ['Trung bình']})], ignore_index=True)
      elif(0.3<=r<=0.5) or (-0.5<=r<=-0.3):
        result_table = pd.concat([result_table, pd.DataFrame.from_dict
         ({'Column': [col], 'pearson-r': [r], 'p-value': [p], 'Tương Quan': ['Yếu']})], ignore_index=True)
      else:
        result_table = pd.concat([result_table, pd.DataFrame.from_dict
         ({'Column': [col], 'pearson-r': [r], 'p-value': [p], 'Tương Quan': ['Không tương quan']})], ignore_index=True)
    else:
      result_table = pd.concat([result_table, pd.DataFrame.from_dict
       ({'Column': [col], 'pearson-r': [r], 'p-value': [p], 'Tương Quan': ['Không tương quan']})], ignore_index=True)
  return result_table

corr2(x)
```

	Column	pearson-r	p-value	Tương Quan
0	year_of_birth	-0.125032	1.087224e-13	Không tương quan

Hình 17: Hệ số tương quan Spearman

2.5 Khai phá tri thức

Khai phá tri thức là việc xác định các mẫu, xu hướng và mối quan hệ ẩn trong dữ liệu nhằm mục đích trích xuất thông tin hữu ích từ các tập dữ liệu lớn và sử dụng chúng để đưa ra dự đoán hoặc ra quyết định tốt hơn. Trong thời kỳ mà thông tin càng lúc càng nhiều, dẫn đến sự bùng nổ dữ liệu thì việc khai phá tri thức càng trở nên quan trọng.

Trong phần này, nhóm tìm hiểu về luật kết hợp trong khai phá tri thức và áp dụng thuật toán Apriori để tìm ra các luật kết hợp từ bộ dữ liệu MOOCCubeX.

2.5.1 Luật kết hợp

Luật kết hợp là một dạng quy tắc mô tả mối liên hệ giữa các tập thuộc tính trong một tập dữ liệu và biểu diễn dưới dạng mệnh đề kéo theo: $X => Y$.

Ta có các khái niệm sau:

- Gọi $I = x_1, x_2, \ldots, x_n$ là một tập các item. Chẳng hạn như tập các sản phẩm trong cửa hàng là $I = \{$sữa, trứng, đường, mật ong, mứt, bơ, mỳ tôm, $\ldots\}$.
- Một tập X gồm các item là tập con của I được gọi là một **itemset**. Một itemset gồm k items gọi là k-itemset
- Ký hiệu $D = T_1, T_2, \ldots, T_m$ là **cơ sở dữ liệu giao dịch** gồm m **giao dịch**. Mỗi giao dịch $T_i \in D$ là một itemset và $T_i \subseteq I$.

Hình 18: Ví dụ về cơ sở dữ liệu giao dịch

Một ví dụ về luật kết hợp là ta có thể giả sử: X = {Sandwich, thịt xông khói, mayonnaise} và Y = {Cà chua} và ta có luật kết hợp $X => Y$ thì có thể nói rằng những người mua sandwich, thịt xông khói và mayonnaise thì cũng thường mua cà chua, có lẽ là để làm hamburger. Tóm lại, mục đích của việc tìm kiếm các luật kết hợp là tìm ra sự kết hợp (association) hay tương quan (correlation) giữa các item trong dữ liệu.

Để tìm ra luật kết hợp trong dữ liệu, ta có thể sử dụng một số các tham số đo lường luật kết hợp. Nhóm nghiên cứu 2 tham số để đo lường luật kết hợp: Độ hỗ trợ và độ tin cậy.

2.5.2 Đo lường luật kết hợp

Độ hỗ trợ (Support) của luật kết hợp $X => Y$ là tần suất của giao dịch chứa tất cả các items trong cả hai tập X và Y. Ví dụ, support của luật $X => Y$ là 5% có nghĩa là 5% các giao dịch X và Y được mua cùng nhau.

Công thức để tính support của luật $X => Y$ như sau:

$$\text{support}(X \Rightarrow Y) = \frac{\text{count}(X \cup Y)}{\text{số lượng các giao dịch}}$$

Trong đó:

- count$(X \cup Y)$ là số lần mà cả X và Y xuất hiện cùng nhau trong các giao dịch. Trong tập dữ liệu, đây là số lượng mẫu dữ liệu chứa cả X và Y.
- Tổng số các giao dịch, giá trị này cũng chính là tổng số mẫu dữ liệu.

Độ tin cậy (Confidence) của luật kết hợp $X => Y$ là xác suất xảy ra Y khi đã biết X. Ví dụ độ tin cậy của luật kết hợp sữa => trứng là 80% có nghĩa là 80% khách hàng mua sữa cũng mua trứng.

Công thức để tính độ tin cậy của luật kết hợp $X => Y$ là xác suất có điều kiện Y khi đã biết X như sau:

$$\text{confidence}(X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$$

Trong đó: support(X) là tỷ lệ số lượng các giao dịch chứa X và tổng số giao dịch.

Để thu được các luật kết hợp, ta thường áp dụng 2 tiêu chí: minimum support (min_sup) và minimum confidence (min_conf). 2 giá trị đó là các giá trị ngưỡng (threshold) và phải xác định trước. Các luật thỏa mãn có support và confidence lớn hơn hoặc bằng cả min_sup và min_conf gọi là các luật kết hợp mạnh. Một itemsets mà tần suất xuất hiện của nó >= min_sup goi là **frequent itemsets**.

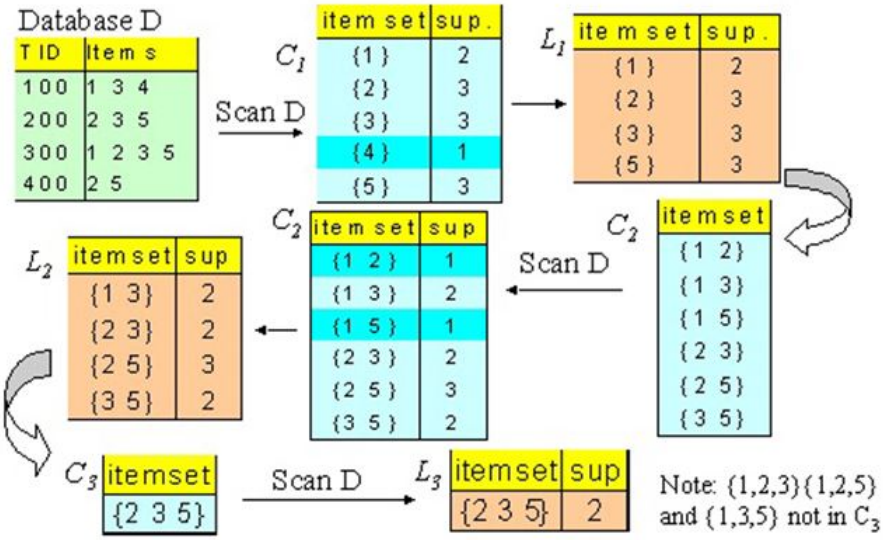
2.5.3 Thuật toán Apriori

Một trong những thuật toán phổ biến nhất tìm các luật kết hợp là Apriori, tư tưởng chính của thuật toán Apriori là:

- Tìm tất cả frequent itemsets: k-itemset (itemsets gồm k items) được dùng để tìm (k+1)- itemset. Đầu tiên tìm 1-itemset (ký hiệu L_1). L_1 được dùng để tìm L_2 (2-itemsets). L_2 được dùng để tìm L_3 (3-itemset) và tiếp tục cho đến khi không có k-itemset được tìm thấy.
- Từ frequent itemsets sinh ra các luật kết hợp mạnh (các luật kết hợp thỏa mãn 2 tham số min_sup và min_conf).

Dưới đây là trình tự thực hiện của thuật toán Apriori:

- Duyệt toàn bộ cơ sở dữ liệu giao dịch để có được support S của 1-itemset, so sánh S với min_sup, để có được 1-itemset (L_1).
- Sử dụng phép kết L_{k-1} với L_{k-1} để sinh ra candidate k-itemset. Loại bỏ các itemsets không phải là frequent itemsets thu được k-itemset.
- Duyệt cơ sở dữ liệu giao dịch để có được support của mỗi candidate k-itemset, so sánh S với min_sup để thu được frequent k –itemset (L_k).
- Lặp lại từ bước 2 cho đến khi Candidate set (C) trống (không tìm thấy frequent itemsets)
- Với mỗi frequent itemset I, sinh tất cả các tập con s không rỗng của I.
- Với mỗi tập con s không rỗng của I, sinh ra các luật s => (I - s) nếu độ tin cậy của nó >= min_conf.



Hình 19: Ví dụ minh họa cách hoạt động của thuật toán Apriori

Quay trở lại với bộ dữ liệu MOOCCubeX, nhóm sẽ sử dụng thuật toán Apriori để phát hiện mẫu và quy tắc kết hợp trong dữ liệu user-xiaomu.json chứa đựng các câu hỏi với chatbot Xiaomu của những người học.

```
user_xiaomu_df = pd.read_json('relations/user-xiaomu.json', lines = True)
```

```
user_xiaomu_df
```

	user_id	question_type	question
0	U_2480182	预设问答	查询课程资源
1	U_2480182	预设问答	推荐学术资源
2	U_2480182	预设问答	解答平台常见问题
3	U_2480182	预设问答	推荐学术资源
4	U_2480182	作诗	休闲娱乐
...	...	...	...
108346	U_37237653	知识图谱点击	相对论
108347	U_37237653	预设问答	查看学点成绩报告
108348	U_37242696	知识图谱点击	全图
108349	U_36604250	预设问答	解释知识概念
108350	U_36604250	来自mooccube|视频答案	二叉树的概念是什么

108351 rows × 3 columns

Hình 20: Dữ liệu của user-xiaomu.json

Ta tìm kiếm luật kết hợp nhị phân trên 2 thuộc tính *user_id* và *question_type*. Ví dụ với một user X và question_type Y, luật kết hợp X => Y nghĩa là một user X thường hỏi các câu hỏi thuộc thể loại Y. Chọn 2 giá trị min_sup và min_conf đều là 0.05. Áp dụng thuật toán Apriori sử dụng thư viện mlxtend - một thư viện chứa các công cụ hỗ trợ cho khoa học dữ liệu.

```
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules

user_xiaomu_df['user_id'] = user_xiaomu_df['user_id'].astype('category')
user_xiaomu_df['question_type'] = user_xiaomu_df['question_type'].astype('category')
transactions = pd.get_dummies(user_xiaomu_df[['user_id', 'question_type']])

# Apply Apriori algorithm to find frequent itemsets
frequent_itemsets = apriori(transactions, min_support=0.05, use_colnames=True)

# Find association rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.05)

print("Frequent Itemsets:")
print(frequent_itemsets)
print("\nAssociation Rules:")
print(rules)
```

```
Frequent Itemsets:
    support                                itemsets
0  0.066294                            (user_id_U_)
1  0.119547              (question_type_来自mooccube)
2  0.059307                 (question_type_来自图灵机器人)
3  0.469613                  (question_type_知识图谱点击)
4  0.144946                    (question_type_预设问答)
5  0.055413  (question_type_来自mooccube, user_id_U_)

Association Rules:
                 antecedents                 consequents  antecedent support  \
0  (question_type_来自mooccube)                (user_id_U_)            0.119547
1               (user_id_U_)  (question_type_来自mooccube)            0.066294

   consequent support   support  confidence      lift  leverage  conviction  \
0            0.066294  0.055413    0.463522  6.991935  0.047487    1.740437
1            0.119547  0.055413    0.835862  6.991935  0.047487    5.364119

   zhangs_metric
0       0.973337
1       0.917824
```

Hình 21: Output khi áp dụng thuật toán Apriori

Kết quả từ thuật toán Apriori bao gồm thông tin về các tập phổ biến và các luật kết hợp tìm thấy từ dữ liệu. Có các itemset phổ biến như sau:

- user_id_U_ với support 0.066294.

- question_type_mooccube với support 0.119547.
- question_type_ với support 0.059307.
- question_type_ với support 0.469613.
- question_type_ với support 0.144946.
- (question_type_mooccube, user_id_U_) với support 0.055413.

Các itemset được tìm thấy là các item xuất hiện nhiều trong tập dữ liệu.

Có hai luật kết hợp được tìm thấy:

- Luật 1: Khi question_type_mooccube xuất hiện, thì user_id_U_ cũng xuất hiện (support: 0.055413, confidence: 0.463522)
- Luật 2: Khi user_id_U_ xuất hiện, thì question_type_mooccube cũng xuất hiện (support: 0.055413, confidence: 0.835862)

Về itemset user_id: 'U_' có 6.6294% các itemsets có chứa U_.

```
result = user_xiaomu_df[user_xiaomu_df['user_id'] == 'U_']
print(result)
```

```
       user_id question_type  \
359         U_       来自图灵机器人
423         U_       来自图灵机器人
539         U_    来自mooccube
540         U_    来自mooccube
541         U_    来自mooccube
...        ...           ...
102564      U_       来自图灵机器人
102799      U_       来自baidu
103712      U_       来自图灵机器人
103713      U_       来自图灵机器人
105367      U_       来自baidu

                                        question
359                                  1234567890-=】
423                                           ·11
539                                       数理逻辑
540                                         属于
541                                    透射电子显微镜
...                                           ...
102564                                       签7
102799                                        、
103712                                      +=、
103713                                      ===
105367  C                        CCC    C CCCCCCCCC...

[7183 rows x 3 columns]
```

Hình 22: Output khi áp dụng thuật toán Apriori

Nhóm nhận thấy một điều khá kỳ lạ khi theo document mà nhóm tác giả của MOOCCubeX có đề rằng user_id sẽ có ký hiệu U_ + [con số nào đó]. Tuy nhiên, việc U_ xuất hiện chỉ ra rằng có một số các user_id đã được gán nhãn không hợp lý. Nhóm thực hiện câu lệnh để chứng minh luận điểm trên.

Hình 22 cho thấy có tận 7183 bản ghi có user_id không đúng định dạng và khẳng định cho điều bất thường mà nhóm nhận thấy.

3 Làm sạch dữ liệu

3.1 Xử lý dữ liệu trùng lặp

Là bước xử lý các dữ liệu có sự lặp lại về giá trị của các trường hoặc các cột trong bộ dữ liệu. Việc này giúp làm tăng hiệu suất hệ thống, tăng tốc quá trình xử lý bộ dữ liệu, không mất thời gian vào những dữ liệu bị trùng lặp.

Xét trên tập dữ liệu user-problem.json với 10000 mẫu dữ liệu:

```
user_problem_df
```

	log_id	problem_id	user_id	is_correct	attempts	score	submit_time
0	100006906522	Pm_6906522	U_10000	0	1	NaN	2020-10-27 10:11:56
1	100006906523	Pm_6906523	U_10000	0	1	NaN	2020-10-27 10:12:13
2	100006906524	Pm_6906524	U_10000	1	1	NaN	2020-10-27 10:12:28
3	100006906525	Pm_6906525	U_10000	0	1	NaN	2020-10-27 10:14:56
4	100006906526	Pm_6906526	U_10000	0	1	NaN	2020-10-27 10:15:18
...	...	...	...	...	...	...	...
9995	100133143580433	Pm_3580433	U_10013314	0	1	35.0	2020-06-28 23:51:49
9996	100133143623001	Pm_3623001	U_10013314	0	1	25.0	2020-06-29 00:12:45
9997	100133143623002	Pm_3623002	U_10013314	0	1	25.0	2020-06-29 00:22:31
9998	100133143707285	Pm_3707285	U_10013314	0	1	30.0	2020-06-29 00:33:23
9999	100133143707286	Pm_3707286	U_10013314	0	1	15.0	2020-06-29 00:43:38

Hình 23: Dữ liệu của user-problem.json với 10000 mẫu dữ liệu

Việc xử lý dữ liệu trùng lặp được thực hiện qua các bước sau:

- Xác định: Sử dụng các kỹ thuật thống kê hoặc học máy để xác định các điểm dữ liệu trùng lặp.
- Loại bỏ: Xoá bỏ các điểm dữ liệu trùng lặp. Việc xác định và xoá bỏ dữ liệu trùng lặp được thực hiện bằng thư viện Pandas trên ngôn ngữ Python qua lệnh df.duplicated() cho việc xác định dữ liệu trùng lặp và df.drop_duplicates() cho việc xoá bỏ dữ liệu trùng lặp trong bộ dữ liệu.

Áp dụng kỹ thuật xác định mẫu dữ liệu trùng lặp df.duplicated() và xoá bỏ dữ liệu trùng lặp df.drop_duplicates():

```
user_problem_df[user_problem_df.duplicated()]
```

log_id	problem_id	user_id	is_correct	attempts	score	submit_time

Không có dữ liệu nào trùng lặp. Tuy nhiên, giả dụ có object nào trùng lặp, ta thực hiện câu lệnh bên dưới

```
user_problem_df.drop_duplicates()
```

	log_id	problem_id	user_id	is_correct	attempts	score	submit_time
0	100006906522	Pm_6906522	U_10000	0	1	NaN	2020-10-27 10:11:56
1	100006906523	Pm_6906523	U_10000	0	1	NaN	2020-10-27 10:12:13
2	100006906524	Pm_6906524	U_10000	1	1	NaN	2020-10-27 10:12:28
3	100006906525	Pm_6906525	U_10000	0	1	NaN	2020-10-27 10:14:56
4	100006906526	Pm_6906526	U_10000	0	1	NaN	2020-10-27 10:15:18
...	...	...	...	...	...	...	...
9995	100133143580433	Pm_3580433	U_10013314	0	1	35.0	2020-06-28 23:51:49
9996	100133143623001	Pm_3623001	U_10013314	0	1	25.0	2020-06-29 00:12:45
9997	100133143623002	Pm_3623002	U_10013314	0	1	25.0	2020-06-29 00:22:31
9998	100133143707285	Pm_3707285	U_10013314	0	1	30.0	2020-06-29 00:33:23
9999	100133143707286	Pm_3707286	U_10013314	0	1	15.0	2020-06-29 00:43:38

10000 rows × 7 columns

Hình 24: Xác định và xoá bỏ dữ liệu trùng lặp trên tập dữ liệu user-problem.json với 10000 mẫu dữ liệu

Lệnh df.duplicated() trả về kết quả dạng boolean trên từng mẫu dữ liệu, mẫu dữ liệu được gán nhãn là True khi dữ liệu này bị trùng lặp trong tập dữ liệu và ngược lại mẫu dữ liệu được gán nhãn là False khi dữ liệu này không bị trùng lặp.

Ở đây ta có thể thấy df.duplicated() trả ra các giá trị False ở 5 mẫu dữ liệu đầu và 5 mẫu dữ liệu cuối. Để hiển thị ra rõ hơn các dữ liệu nào bị trùng lặp, ta dùng df[df.duplicated()] để hiển thị các mẫu dữ liệu được gán nhãn là True. Quan sát ở hình 24, ta thấy không có dữ liệu nào được trả về qua câu lệnh df[df.duplicated()] cho thấy trong 10000 mẫu dữ liệu từ user-problem.json không có dữ liệu bị trùng lặp. Để minh hoạ rõ hơn, qua lệnh xoá bỏ dữ liệu trùng lặp df.drop_duplicates(), ta thấy số mẫu dữ liệu sau khi xử lý việc xoá bỏ dữ liệu trùng lặp tương đương với tập dữ liệu ban đầu chưa qua xử lý.

4 Tài liệu tham khảo

[1] Chuẩn bị dữ liệu - ThS. Nguyễn Thị Anh Thư

[2] Thuật toán Apriori khai phá luật kết hợp - Nguyễn Văn Chức

[3] Khai phá mẫu phổ biến và luật kết hợp - Nguyễn Văn Hiếu